

Einführung in C - Teil 1

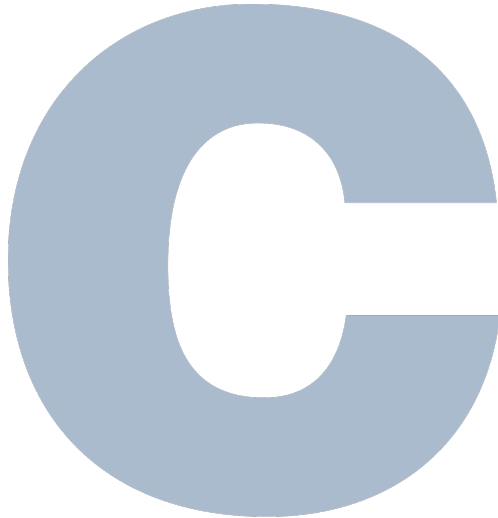


Figure 1: C-Logo

Geschichte von C

- ▶ Anfang der 1970er Jahre von Dennis Ritchie bei Bell Labs entwickelt.
- ▶ Entstand als Weiterentwicklung der Sprache B.
- ▶ Wurde hauptsächlich für die Entwicklung des Betriebssystems Unix genutzt.

- ▶ 1978 erschien das einflussreiche Buch The C Programming Language.

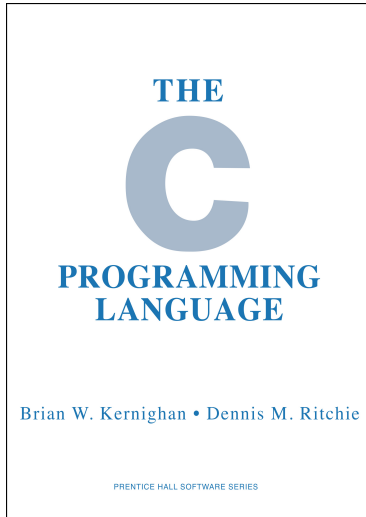


Figure 2: The C Programming Language

- ▶ 1989 wurde C als ANSI-Standard normiert (ANSI C).
- ▶ Bis heute eine der wichtigsten Programmiersprachen weltweit.

Warum ist C wichtig?

- ▶ Grundlage vieler moderner Programmiersprachen (z. B. C++, Java, C#).
- ▶ Wird für Betriebssysteme, Treiber und eingebettete Systeme verwendet.
- ▶ Sehr effizient und schnell in der Ausführung.
- ▶ Bietet direkten Zugriff auf Speicher und Hardware.

- Viele wichtige Softwareprojekte wurden in C entwickelt.

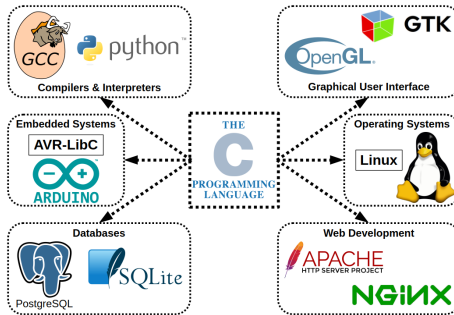


Figure 3: C-Projekte

Warum ist C eine gute Wahl für Programmierneinsteiger?

- ▶ Vermittelt grundlegende Programmierkonzepte sehr klar.
- ▶ Hilft, den Umgang mit Speicher und Daten besser zu verstehen.
- ▶ Relativ kleine und überschaubare Sprachsyntax.
- ▶ Fördert sauberes und strukturiertes Programmieren.
- ▶ Erleichtert später das Lernen anderer Programmiersprachen.
- ▶ Weit verbreitet und in vielen Lehrbüchern sowie Kursen vertreten.

Grundstruktur eines C-Programms

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello, world!\n");
    return (0);
}
```

1. Einbinden des Headerfiles `stdio.h` für Standard Ein- und Ausgabe-Funktionen wie `printf()`
2. Definition der `main()`-Funktion. Hier beginnt die Ausführung des Programms.
3. Ausgabe von `Hello, world!` auf das Terminal.
4. Verlassen der `main()`-Funktion mit Rückgabewert 0 an den Aufrufer.

Die `main()`-Funktion

- ▶ Jedes Programm braucht einen Startpunkt.
- ▶ Mit der Definition der `main()`-Funktion sagt man dem Betriebssystem, wo die Ausführung startet.
- ▶ `main()` kann optional Parameter haben, z.B. Kommandozeilenargumente.

Rückgabewert von main()

- ▶ Jedes Programm, mit Ausnahme von wenigen Systemprogrammen, wird von einem anderen Prozess ausgeführt.
- ▶ Wir starten unser Programm z.B., indem wir es von ...
 - ▶ ... einem Kommandointerpreter aus im Terminal ausführen
 - ▶ ... einem Dateimanager aus ausführen
 - ▶ ... einer IDE aus ausführen
 - ▶ etc.

- ▶ Den Rückgabewert von `main()` erhält der ausführende Prozess.
- ▶ Dieser kann aus dem Wert Rückschlüsse ziehen, ob z.B. ein Fehler aufgetreten ist.
- ▶ Die fehlerlose Beendigung wird üblicherweise mit einem Wert von 0 signalisiert.

Quelltext-Kommentare

- ▶ Kommentare dienen dazu, den Quellcode für Menschen verständlicher zu machen.
- ▶ Der Compiler ignoriert Kommentare vollständig.
- ▶ In C gibt es zwei Arten von Kommentaren

Blockkommentare

Dieser Kommentar kann sich über eine oder mehrere Zeilen erstrecken.

```
/* Das ist ein Kommentar */
```

```
int x = 5;
```

```
/*
```

```
Dieser Kommentar  
geht über mehrere  
Zeilen.
```

```
*/
```

Zeilenkommentare

Dieser Kommentar beginnt mit `//` und endet automatisch am Zeilenende.

```
int x = 5;  // Variable x deklarieren und initialisieren  
  
// Dies ist eine ganze Kommentarzeile  
printf("%d\n", x);
```

Variablen

- ▶ Fast jedes Programm braucht Variablen zum Zwischenspeichern von Zuständen und Daten.
- ▶ Eine Variable ist ein benannter, begrenzter Bereich im Speicher.
- ▶ Variablen sind in C an einen Typ gebunden.

```
char buchstabe = 'A';  
int zaehler = 0;
```

Variablen-Namen

- ▶ Bestehen aus den Zeichen der Zeichenmenge [a-zA-Z0-9_]
- ▶ Müssen mit einem Buchstaben oder _ beginnen.
- ▶ Dürfen keine C-Schlüsselwörter sein.
- ▶ Sind *case-sensitive*. Groß-/Kleinschreibung ist wichtig!

```
int ungültig;      // Fehler. Enthält den Umlaut ü.  
int 2x;           // Fehler. Beginnt mit einer Ziffer.  
int x2;           // Ok  
int das_ist_ok;   // Snake case. Ok  
int auchDasIstOk; // Camel case. Ok
```


Variablen-Typen

Die wichtigsten Basistypen in C

Typ	Beschreibung	Größe in Bytes
char	wird häufig für einzelne Zeichen verwendet	1
short	Wird für kleine Ganzzahlen verwendet	2
int	Standardtyp für ganze Zahlen, z. B. Zähler, Schleifenvariablen.	≥ 2 (4)
long	Größere ganze Zahlen, z. B. Zeitstempel oder große Zählwerte.	≥ 4 (8)
long	Sehr große ganze Zahlen, z. B.	≥ 8
long	Dateigrößen oder große mathematische Werte	

Die wichtigsten Basistypen in C (Fs.)

Typ	Beschreibung	Größe in Bytes	Stellen
float	Kommazahlen mit mittlerer Genauigkeit, z. B. Messwerte, Temperaturen	4	≥ 6
double	Kommazahlen mit höherer Genauigkeit, z. B. wissenschaftliche Berechnungen	8	≥ 10
long double	Sehr genaue Fließkommaberechnungen	8, 12 oder 16	min. wie double

Beispiel

Mithilfe des sizeof-Operators können wir uns die Größe der Typen ausgeben lassen.

```
#include <stdio.h>

int main() {
    printf("char: %d\n", sizeof(char));
    printf("short: %d\n", sizeof(short));
    printf("int: %d\n", sizeof(int));
    printf("long: %d\n", sizeof(long));
    printf("long long: %d\n", sizeof(long long));
    printf("float: %d\n", sizeof(float));
    printf("double: %d\n", sizeof(double));
    printf("long double: %d\n", sizeof(long double));
}
```

Ausgabe auf einem 64-Bit-System

char: 1

short: 2

int: 4

long: 8

long long: 8

float: 4

double: 8

long double: 16

Deklaration und Initialisierung

- ▶ Variablen müssen in C erst deklariert werden, bevor man sie nutzen kann.
- ▶ Variablen können gleichzeitig deklariert und initialisiert werden.

```
int x;      // Deklaration
```

```
int y = 0;  // Deklaration mit Initialisierung
```

Deklarations-Spezifizierer

`const`, `static` und `unsigned` sind Schlüsselwörter, die unterschiedliche Aspekte einer Variablen-Deklaration in C beeinflussen.

const

- ▶ Macht das Objekt *read-only*: sein Wert darf nach der Initialisierung nicht mehr verändert werden.
- ▶ Ermöglicht Optimierungen, weil der Compiler von einem unveränderlichen Wert ausgehen kann.
- ▶ Ist ein Typ-Qualifizierer und kann mit fast jedem Datentyp kombiniert werden (z.B. `const int`, `const char *`).

```
const int answer = 42;  
answer = 43; // Fehler. answer ist unveränderbar
```

unsigned

- ▶ Typ-Spezifizierer für ganzzahlige Typen ohne Vorzeichen, z.B. `unsigned int`, `unsigned long`, `unsigned char`.
- ▶ Verschiebt den darstellbaren Bereich zu höheren, ausschließlich nichtnegativen Werten (0 bis Maximalwert).
- ▶ Wird häufig für Zähler, Bitmasken und Größen verwendet, bei denen negative Werte keinen Sinn ergeben.

```
unsigned int counter = 0;
```


static

- ▶ Innerhalb einer Funktion: Variable behält ihren Wert zwischen Funktionsaufrufen (*statische Lebensdauer*).
- ▶ Außerhalb von Funktionen (auf Dateiebene): beschränkt die Sichtbarkeit auf diese Übersetzungseinheit (interne Bindung).
- ▶ Wird nur einmal initialisiert, vor Programmbeginn.

```
int next_odd() {  
    static int n = 1;  
    n += 2;  
    return (n - 2);  
}
```

...

```
printf("%d\n", next_odd()); // Ausgabe: 1  
printf("%d\n", next_odd()); // Ausgabe: 3
```

Scope (Gültigkeitsbereich) von Variablen

- ▶ Scope bezeichnet den Bereich im Programm, in dem eine Variable, Funktion oder ein anderer Bezeichner sichtbar und verwendbar ist.
- ▶ Eine Variable kann nur innerhalb ihres Scopes direkt verwendet werden.
- ▶ Außerhalb ihres Scopes ist der Bezeichner unbekannt und führt zu einem Compilerfehler.

Block Scope

- ▶ Variablen, die innerhalb eines Blocks { ... } deklariert werden, haben **Block Scope**.
- ▶ Sie sind nur innerhalb dieses Blocks und seiner Unterblöcke sichtbar.

```
{  
    int x = 5;  
    printf("%d", x); // erlaubt  
}  
printf("%d", x);    // Fehler
```

Globaler Scope

- ▶ Globale Variablen werden außerhalb aller Funktionen deklariert.
- ▶ Sie sind ab ihrer Deklaration bis zum Ende der Datei sichtbar.

```
int global;
```

```
int main() {  
    global = 5;  
}
```

Standard-Werte von Variablen

- ▶ Der Wert einer nicht initialisierten, lokalen Variable ist undefiniert.
- ▶ Der Wert einer statischen oder globalen Variable ist standardmäßig 0.

```
int global; // global = 0
```

```
int main() {  
    int x;    // Undefiniert. Zufälliger Wert  
}
```

Arithmetische Operatoren

Operator	Beispiel	Beschreibung
*	$a * b$	Multiplikation
/	a / b	Division
%	$a \% b$	Modulo/Divisionsrest von a/b
+	$a + b$	Addition
-	$a - b$	Subtraktion

Präzedenz der arithmetischen Operatoren

- ▶ `*`, `/` und `%` haben dieselbe Präzedenz
- ▶ Danach folgen `+` und `-`
- ▶ Klammern (...) überschreiben immer die Rangfolge

```
int a = 3, b = 5, c;  
c = a + a * b;    // c = 18  
c = (a + a) * b;  // c = 30
```

Zeichenrepräsentation in C

- ▶ Zeichen werden intern als Ganzzahlen gespeichert.
- ▶ Der Datentyp `char` belegt in der Regel 1 Byte (8 Bit).
- ▶ Jedes Zeichen besitzt einen numerischen Code.
- ▶ Das Zeichen `'A'` entspricht dem Wert 65, `'a'` dem Wert 97.

ASCII-Tabelle

- ▶ ASCII (American Standard Code for Information Interchange) ordnet Zeichen Zahlenwerte zu.
- ▶ Der Standard umfasst 128 Zeichen (0–127).
- ▶ Enthält:
 - ▶ Steuerzeichen (z. B. Zeilenumbruch \n)
 - ▶ Ziffern (0–9)
 - ▶ Groß- und Kleinbuchstaben (A–Z, a–z)
 - ▶ Sonderzeichen (z. B. !, ?, @)

ASCII Table

0 0x00 NUL	1 0x01 SOH	2 0x02 STX	3 0x03 ETX	4 0x04 EOT	5 0x05 ENQ	6 0x06 ACK	7 0x07 BEL	8 0x08 BS	9 0x09 HT	10 0x0A LF	11 0x0B VT	12 0x0C FF	13 0x0D CR	14 0x0E SO	15 0x0F SI
16 0x10 DLE	17 0x11 DC1	18 0x12 DC2	19 0x13 DC3	20 0x14 DC4	21 0x15 NAK	22 0x16 SYN	23 0x17 ETB	24 0x18 CAN	25 0x19 EM	26 0x1A SUB	27 0x1B ESC	28 0x1C FS	29 0x1D GS	30 0x1E RS	31 0x1F US
32 0x20 0101 0000 SPACE	33 0x21 !	34 0x22 "	35 0x23 #	36 0x24 \$	37 0x25 %	38 0x26 &	39 0x27 '	40 0x28 (	41 0x29)	42 0x2A *	43 0x2B +	44 0x2C ,	45 0x2D -	46 0x2E .	47 0x2F /
48 0x30 0101 0001 0	49 0x31 1	50 0x32 2	51 0x33 3	52 0x34 4	53 0x35 5	54 0x36 6	55 0x37 7	56 0x38 8	57 0x39 9	58 0x3A :	59 0x3B ;	60 0x3C <	61 0x3D =	62 0x3E >	63 0x3F ?
64 0x40 0100 0000 @	65 0x41 A	66 0x42 B	67 0x43 C	68 0x44 D	69 0x45 E	70 0x46 F	71 0x47 G	72 0x48 H	73 0x49 I	74 0x4A J	75 0x4B K	76 0x4C L	77 0x4D M	78 0x4E N	79 0x4F O
80 0x50 0101 0000 P	81 0x51 Q	82 0x52 R	83 0x53 S	84 0x54 T	85 0x55 U	86 0x56 V	87 0x57 W	88 0x58 X	89 0x59 Y	90 0x5A Z	91 0x5B [	92 0x5C]	93 0x5D ^	94 0x5E _	95 0x5F `
96 0x60 0110 0000 a	97 0x61 b	98 0x62 c	99 0x63 d	100 0x64 e	101 0x65 f	102 0x66 g	103 0x67 h	104 0x68 i	105 0x69 j	106 0x6A k	107 0x6B l	108 0x6C m	109 0x6D n	110 0x6E o	111 0x6F p
112 0x70 0111 0000 q	113 0x71 r	114 0x72 s	115 0x73 t	116 0x74 u	117 0x75 v	118 0x76 w	119 0x77 x	120 0x78 y	121 0x79 z	122 0x7A {	123 0x7B 	124 0x7C }	125 0x7D ~	126 0x7E DEL	127 0x7F DEL

Figure 4: ASCII-Tabelle

In C können Zeichen über ihre ASCII-Werte verglichen oder berechnet werden.

Beispiel

```
char c = 'A';  
printf("%d", c); // Ausgabe: 65
```

C-Steuerzeichen

- ▶ C-Steuerzeichen sind spezielle Escape-Sequenzen in String-/Char-Literalen.
- ▶ Start mit \ gefolgt von einem Zeichen bzw. einer Sequenz.

Zeichen	Beschreibung
\n	Zeilenumbruch
\t	Tabulator
\r	Wagenrücklauf
\b	Backspace
\f	Seitenvorschub (Formfeed)
\a	Alarm (Glocke)
\\	Backslash
\'	einfaches Hochkomma
\"	doppeltes Hochkomma
\0	Nullbyte (String-Ende)

Beispiel

```
printf("foo\n\tbar\n\t\tbaz\n");
```

Ausgabe

```
foo
    bar
        baz
```

Die C-Standardbibliothek

- ▶ Die C-Standardbibliothek ist der Teil der Programmbibliotheken, die zu C gehören, und die bei jedem C-Compiler/Plattform vorhanden sein müssen.
- ▶ Sie ist im C-Standard festgelegt (z. B. Datei- und Stringfunktionen, mathematische Funktionen, Ein-/Ausgabe über Standard-Streams).
- ▶ Sie wird typischerweise über Header-Dateien wie `stdio.h`, `stdlib.h`, `string.h`, `math.h` usw. eingebunden.
- ▶ Ziel ist Portabilität: Code, der sich an den Standard hält, soll auf verschiedenen Systemen funktionieren.

stdio.h

stdio.h ist die Header-Datei für die Standard-Ein-/Ausgabe in C (z. B. printf, scanf, fopen, fclose, fread, fwrite, FILE-Streams).

printf()

- ▶ printf() ist eine Funktion für formatierte Ausgabe auf die Standardausgabe (meist die Konsole).
- ▶ Sie nimmt eine Formatzeichenkette und dazu passende Argumente entgegen (Platzhalter wie %d, %f, %s).
- ▶ Gibt die Anzahl der ausgegebenen Zeichen (oder bei Fehlern einen negativen Wert) zurück.

```
int printf(format, arg1, arg2, ...);
```


Wichtige Formatstring-Platzhalter

- ▶ %d: ganze Zahl (int)
- ▶ %u: vorzeichenlose ganze Zahl (unsigned int)
- ▶ %c: einzelnes Zeichen (char)
- ▶ %s: Zeichenkette (String)
- ▶ %f: Fließkommazahl (float, double)
- ▶ %e: Fließkommazahl in Exponentialschreibweise
- ▶ %x: ganze Zahl in Hexadezimaldarstellung
- ▶ %o: ganze Zahl im Oktalsystem
- ▶ %p: Speicheradresse (Pointer)

Beispiel

```
printf("Alter: %d\n", 20);  
printf("Note: %c\n", 'A');  
printf("Pi: %f\n", 3.14159);  
printf("Name: %s\n", "Max");
```

Ausgabe

```
Alter: 20  
Note: A  
Pi: 3.14159  
Name: Max
```

Beispiel

```
printf("Mein Name ist %s und ich bin %d Jahre alt", \
      "Max", 20);
```

Ausgabe

Mein Name ist Max und ich bin 20 Jahre alt

scanf()

- ▶ `scanf()` ist eine Funktion für formatierte Eingabe von der Standard-Eingabe (meist Tastatur).
- ▶ Sie verwendet eine Formatzeichenkette mit Platzhaltern wie `%d`, `%f`, `%s` und passende Variablen als Ziele.
- ▶ Bei den meisten Datentypen muss die Adresse der Variablen mit `&` übergeben werden.
- ▶ Sie liest Daten nach dem Format ein und gibt zurück, wie viele Werte erfolgreich gelesen wurden (oder EOF bei Fehler/Ende).

```
int scanf(format, arg1, arg2, ...);
```

Wichtige Formatstring-Platzhalter

- ▶ %d: ganze Zahl (`int`)
- ▶ %u: vorzeichenlose ganze Zahl (`unsigned int`)
- ▶ %c: einzelnes Zeichen (`char`)
- ▶ %s: Zeichenkette (`String`)
- ▶ %f: Fließkommazahl (`float`)
- ▶ %lf: Fließkommazahl (`double`)
- ▶ %x: Hexadezimalzahl
- ▶ %o: Oktalzahl

Beispiel

```
int alter;  
float temperatur;  
char note;
```

```
scanf("%d", &alter);  
scanf("%f", &temperatur);  
scanf("%c", &note);
```

Beispiel

`scanf()` kann auch mehrere durch *White spaces* getrennte Werte einlesen.

```
int a, b;
```

```
scanf("%d %d", &a, &b);
```

Beispiel

Der Rückgabewert dient der Fehlererkennung.

```
int a, n;

print("Eine Ganzzahl eingeben: ");
n = scanf("%d", &a);

if (n != 1) {
    printf("Fehler: Bitte eine Ganzzahl eingeben.");
}
```

Ausgabe

Eine Ganzzahl eingeben: xyz

Fehler: Bitte eine Ganzzahl eingeben.

Verzweigungen mit if, else und else if

- ▶ Dienen zur Fallunterscheidung und Programmsteuerung.
- ▶ Eine Bedingung wird geprüft und abhängig vom Ergebnis wird unterschiedlicher Code ausgeführt.
- ▶ if führt einen Codeblock aus, wenn die Bedingung wahr (ungleich 0) ist.

```
if (alter >= 18) {  
    printf("Volljährig");  
}
```

- ▶ else führt einen Codeblock aus, wenn die Bedingung des zugehörigen if falsch (0) ist.

```
if (alter >= 18) {  
    printf("Volljährig");  
} else {  
    printf("Minderjährig");  
}
```

- ▶ `else if` prüft eine weitere Bedingung, wenn die vorherige Bedingung falsch (0) war.
- ▶ Beliebige viele `else if`-Zweige sind möglich.

```
if (note == 1) {  
    printf("Sehr gut");  
} else if (note == 2) {  
    printf("Gut");  
} else {  
    printf("Andere Note");  
}
```

- ▶ Bedingungen stehen in runden Klammern ().
- ▶ Codeblöcke werden meist mit geschweiften Klammern {} geschrieben.
- ▶ Es wird immer nur **der erste zutreffende Zweig** ausgeführt.
- ▶ else ist optional und steht immer am Ende einer Verzweigungskette.

Vergleichs- und logische Operatoren

- ▶ `if` wertet das Ergebnis eines Booleschen Ausdrucks aus.
- ▶ Boolesche (logische) Ausdrücke sind Ausdrücke, die einen Wahrheitswert (wahr/falsch, 0/1) liefern und aus relationalen und/oder logischen Operationen zusammengesetzt sein können.

Vergleichsoperatoren

Operator	Beispiel	Ergebnis
<code>==</code>	<code>a == b</code>	1, wenn $a = b$, sonst 0
<code>></code>	<code>a > b</code>	1, wenn $a > b$, sonst 0
<code><</code>	<code>a < b</code>	1, wenn $a < b$, sonst 0
<code>>=</code>	<code>a >= b</code>	1, wenn $a \geq b$, sonst 0
<code><=</code>	<code>a <= b</code>	1, wenn $a \leq b$, sonst 0
<code>!=</code>	<code>a != b</code>	1, wenn $a \neq b$, sonst 0

0 = falsch, 1 = wahr

Logische Operatoren

Boolesche Werte können mit **logischen Operatoren** verknüpft werden, um komplexere Bedingungen zu formulieren.

Operator	Beispiel	Ergebnis
&&	a && b	1, wenn a und b wahr sind, sonst 0
	a b	1, wenn a oder b wahr ist, sonst 0
!	!a	1, wenn a 0 ist, sonst 0

Beispiel

```
int alter = 23;
```

```
printf("Ergebnis (alter = %d): %d\n", alter, \
      alter >= 18 && alter < 65);
```

```
alter = 77;
```

```
printf("Ergebnis (alter = %d): %d\n", alter, \
      alter >= 18 && alter < 65);
```

Ausgabe

```
Ergebnis (alter = 23): 1
```

```
Ergebnis (alter = 77): 0
```


Beispiel

```
int note = 2;  
if (note == 1 || note == 2) {  
    printf("Gut gemacht!\n");  
}
```

Ausgabe

Gut gemacht!

Beispiel

```
int fertig = 0;  
  
if (!fertig) {  
    printf("Noch nicht fertig\n");  
}
```

Ausgabe

Noch nicht fertig

Präzedenz relationaler und logischer Operatoren

Relationale Operatoren

- ▶ $>$, $<$, $>=$ und $<=$ haben dieselbe Präzedenz
- ▶ $==$ und $!=$ kommen direkt danach
- ▶ Klammern (...) überschreiben immer die Rangfolge

Logische Operatoren

- ▶ $!$ hat als *unärer* Operator die höchste Präzedenz
- ▶ $\&\&$ bindet stärker als $||$
 - ▶ $a || b \&\& c = a || (b \&\& c)$
- ▶ Klammern (...) überschreiben immer die Rangfolge

Lazy Evaluation

- ▶ Auswertung erst wenn nötig.
- ▶ In C betrifft das vor allem `&&` und `||`.
- ▶ `&&` und `||` sind *linksassoziativ*, d.h., es wird von links nach rechts ausgewertet.
- ▶ `&&`: Wenn links falsch ist (0), wird rechts nicht mehr ausgewertet.
- ▶ `||`: Wenn links wahr ist (nicht 0), wird rechts nicht mehr ausgewertet.

```
// Es wird von links nach rechts ausgewertet  
if (a || b || c && d) {  
    // Wenn a wahr ist, wird b || c && d nicht  
    // ausgewertet  
}
```

Ende

```
printf("Danke für Eure Aufmerksamkeit!\n");  
return (0);
```

Quellen

- ▶ **Cobaiahackers:** *C1stEdition.svg* (C-Logo nach dem Cover der 1. Auflage von *The C Programming Language*), Eigenes Werk, Wikimedia Commons, CC BY-SA 4.0, <https://commons.wikimedia.org/wiki/File:C1stEdition.svg> (abgerufen am 13.07.2026). ([commons.wikimedia.org][1])
- ▶ **Shabaz1000:** *ASCII Table (suitable for printing)*, Eigenes Werk, Wikimedia Commons, CC BY-SA 4.0, https://commons.wikimedia.org/wiki/File:ASCII_Table_%28suitable_for_printing%29.svg (abgerufen am 13.07.2026). ([commons.wikimedia.org][2])

- ▶ **Maxim Pe:** *The C Programming Language.png*, Eigenes Werk, Wikimedia Commons, CC BY-SA 4.0, https://commons.wikimedia.org/wiki/File:The_C_Programming_Language.png (abgerufen am 13.07.2026). ([Wikimedia Commons][1])
- ▶ **Julesmazur:** *The C Programming Language, First Edition Cover.svg*. Abgeleitet vom Buchcover von *The C Programming Language* (Prentice Hall, 1978), Wikimedia Commons, Public Domain (gemäß Wikimedia-Kennzeichnung), abgerufen am 13.07.2026. https://commons.wikimedia.org/wiki/File:The_C_Programming_Language,_First_Edition_Cover.svg. ([Wikimedia Commons][1])